

LC 2653: Sliding Subarray Beauty

Amazon Interview Problem Breakdown & C++ Solutions

Problem Overview

For a fixed-size window of length k , the task is to return the x^{th} smallest negative number. If the x^{th} smallest negative number doesn't exist within that fixed window, return 0 .

Example: Given the array $[1, -1, -3, -2, 3]$, $k = 3$, $x = 2$.

Output: $[-1, -2, -2]$

Approach 1: Using Multiset (Inherently Optimal)

A multiset is used because it allows duplicate values and inherently stores elements in sorted order. We optimize the space and logic by strictly storing only the negative numbers in our multiset.

The insertion and deletion process for the sliding window takes $O(\log k)$. Once the window is updated, finding the x^{th} smallest negative number requires advancing an iterator by x steps, which takes $O(x)$.

Time Complexity: $O(n \times (\log k + x))$ | **Space Complexity:** $O(k)$

```

class Solution {
public:
    vector<int> getSubarrayBeauty(vector<int>& nums, int k, int x) {
        int n = nums.size();
        multiset<int> st;
        vector<int> ans;

        for (int i = 0; i < k; i++) {
            if (nums[i] < 0) {
                st.insert(nums[i]);
            }
        }

        if (st.size() < x) {
            ans.push_back(0);
        }

        else {
            auto it = st.begin();
            for (int i = 1; i < x; i++) {
                it++;
            }
            ans.push_back(*it);
        }

        for (int j = k; j < n; j++) {
            if (nums[j - k] < 0) {
                st.erase(st.find(nums[j - k]));
            }

            if (nums[j] < 0) {
                st.insert(nums[j]);
            }

            if (st.size() < x) {
                ans.push_back(0);
            }

            else {
                auto it = st.begin();
                for (int i = 1; i < x; i++) {
                    it++;
                }
            }
        }
    }
}

```

```
        ans.push_back(*it);
    }
}
return ans;
}
};
```

Approach 2: Frequency Array (Beats 100% - With a Catch)

This approach leverages the problem's specific constraints. The constraints on the values are small (typically covering only **50** to **100** possible negative integer values).

Instead of sorting or using a tree-based map, we can use a frequency array. The frequency mapping can be done in **$O(n)$** . However, to find the **x^{th}** smallest negative number, we scan the frequency array (up to **N** negative numbers) for every window. Hence, if there were **N** possible negative numbers and **n** windows, the complexity becomes **$O(n \times N)$** . We maintain and update the frequency map as the window slides.

Time Complexity: $O(n \times 50)$ | Space Complexity: $O(50)$ constant space

```

class Solution {
public:
    int getBeauty(vector<int>& freq, int x)
    {
        int cnt = 0;

        for(int val = 50; val >= 1; val--)
        {
            cnt += freq[val];

            if(cnt >= x)
                return -val;
        }

        return 0;
    }

    vector<int> getSubarrayBeauty(vector<int>& nums, int k, int x) {
        int n = nums.size();

        vector<int> ans;
        vector<int> freq(51, 0);

        for(int i = 0; i < k; i++)
        {
            if(nums[i] < 0)
                freq[-nums[i]]++;
        }

        ans.push_back(getBeauty(freq, x));

        for(int i = k; i < n; i++)
        {
            if(nums[i-k] < 0)
                freq[-nums[i-k]]--;

            if(nums[i] < 0)
                freq[-nums[i]]++;

            ans.push_back(getBeauty(freq, x));
        }

        return ans;
    }
};

```

```
}  
};
```